

Algorithmic Micro-Donation & Food-Waste Routing Service

Full Project Description

Department of Data Science · Subject: CPI

1. Executive Summary

This project builds a real-time coordination platform that connects surplus food from commercial kitchens, restaurants, and events to non-profit recipients — shelters, food banks, community kitchens — before that food becomes unusable. It is not a listing website where donors post food and hope someone notices. The core of the system is a **constraint-based matching and optimisation engine** that decides, algorithmically, which eligible recipient should receive a given batch of surplus food, whether the transfer is physically feasible before expiry, and how the pickup and delivery should be routed — then tracks that donation from creation through digital sign-off.

The problem being solved is not food scarcity. Surplus food already exists in large quantities, and non-profits willing to receive it already exist. What breaks is **coordination**: shelf life shrinks rapidly once food leaves a kitchen, manual phone-call/WhatsApp matching is too slow, NGOs have limited storage and pickup capacity that nobody is tracking in real time, and transit delays quietly turn edible food into waste. The platform exists to close that coordination gap.

2. Problem Statement

2.1 What actually fails today

- Large quantities of food become surplus with little notice (an event ends early, a kitchen over-prepares, a caterer has leftover stock).
- NGOs have limited storage and pickup capacity, and no one outside the NGO knows what that capacity is at any given moment.

- Shelf life shrinks rapidly once food leaves controlled kitchen conditions.
- Manual recipient-matching — phone calls, WhatsApp groups — is too slow relative to how fast a shelf-life window closes.
- Transit delays render still-edible food unusable by the time it arrives.
- Donors face documentation and liability uncertainty around handing off food to a third party.

2.2 Scale of the problem

- India generates roughly **55 kg of food waste per person per year in households alone** — about 78.2 million tonnes annually (UNEP Food Waste Index Report 2024, 2022 data; confirmed by the Government of India, Lok Sabha Unstarred Question 2181, 12 March 2025).
- The same report contains **no equivalent Indian datapoint for food service or retail** — the sector this project actually targets is measured globally but unmeasured domestically, which is itself part of the case for building tooling here.
- Globally, by sector in 2022: households accounted for **631 million tonnes**, food service **290 million tonnes**, and retail **131 million tonnes** of food waste (UNEP Food Waste Index Report 2024).

2.3 Why this is a coordination problem, not a supply or willingness problem

The recipient pool already exists at meaningful scale: FSSAI's **Indian Food Sharing Alliance** lists **82 registered surplus-food distribution agencies across 90+ cities** (Govt. of India, Lok Sabha USQ 2181, 12 March 2025). The gap is not "no one wants this food" — it's that nobody is running the real-time matching, capacity-awareness, and routing that would let supply actually reach that existing network before it spoils.

3. Proposed Solution

3.1 What the platform does

1. Registers surplus food in real time, as it arises.
2. Identifies eligible non-profits within reach.
3. Weighs distance, transit time, storage capacity, shelf life, and current demand.
4. Generates an optimised pickup and dispatch plan.

5. Digitally records handover and a documentation/traceability sign-off.
6. Tracks each donation from creation through pickup to delivery.

3.2 What makes this different from the status quo

Traditional approach	This platform
Phone calls and WhatsApp groups	Real-time, structured matching
Manual coordination by staff	Automated dispatch workflow
"Nearest NGO" heuristic	Shelf-life-aware allocation
Static maps and fixed routes	Dynamic, traffic-aware routing
No view of NGO storage limits	Capacity-aware matching
No prioritisation between offers	Transit-time optimisation and ranking
Paper or informal handover	Automated digital documentation

The central reframing: the system does not ask *"which NGO is closest?"* It asks *"which feasible recipient minimises the risk of this food becoming unusable, while maximising how much of it is actually used?"* Those are different questions with different answers, and the difference is the entire technical contribution of the project.

3.3 Explicitly NOT a claim being made

The system is **not** an AI/ML matching black box, and the project should never be pitched that way to judges or evaluators. It is real-time optimisation and constraint satisfaction with an explainable, weighted scoring layer on top — every match can be traced back to five named, auditable factors. Machine learning appears only in future scope (demand prediction, spoilage prediction), layered on top of a system that already works without it.

4. System Actors

4.1 Donor

Examples: restaurants, hotels, college canteens, event organisers, commercial kitchens. Creates a surplus-food donation with food type, quantity, preparation time, availability window, expiry/use-by time, pickup location, food category, and special handling

requirements.

4.2 Recipient / NGO

Examples: shelters, food banks, community kitchens, non-profit organisations. Maintains organisation details, location, eligibility, storage capacity, current available capacity, accepted food categories, current demand, operating hours, and pickup capability.

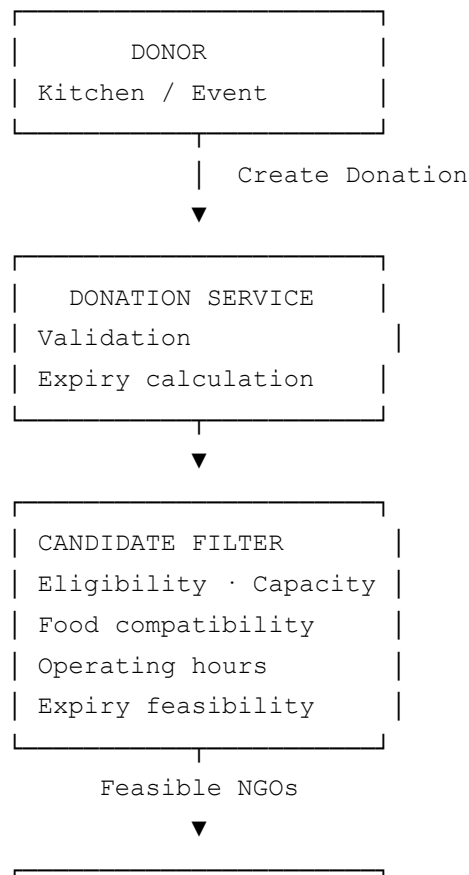
4.3 Driver / Logistics

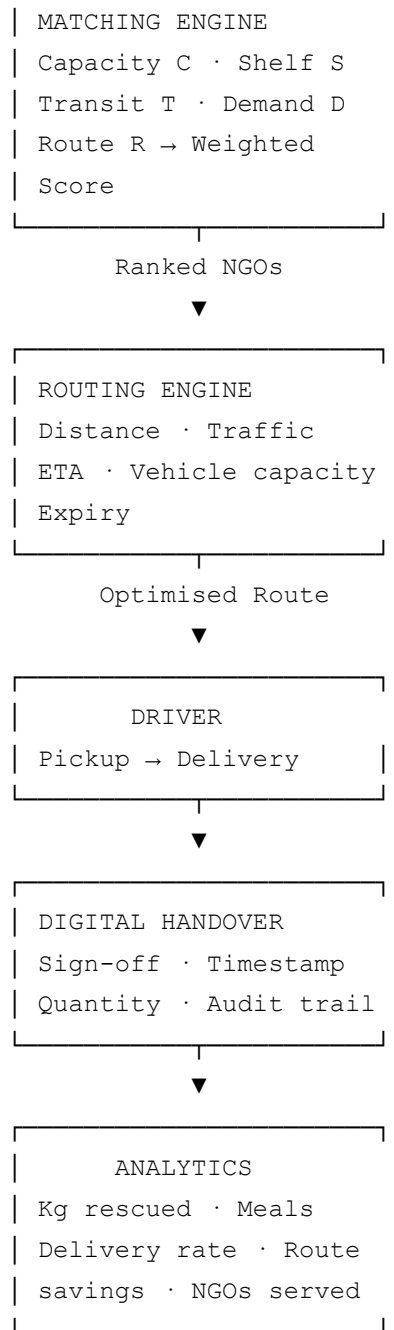
For a prototype, this can be simulated: driver ID, vehicle capacity, current location, availability, current route. Later this becomes a real driver/volunteer marketplace — explicitly future scope.

4.4 Administrator

Manages NGOs, donors, eligibility, food categories, users, reported issues, system parameters, matching weights, and audit logs.

5. System Architecture





The presentation's own architecture is deliberately stack-agnostic — it describes responsibilities, not technologies. Everything from Section 6 onward (FastAPI, React, PostgreSQL/PostGIS, Redis, WebSockets, JWT) is a concrete engineering choice made to actually implement that architecture, not something the brief mandates.

6. Technology Stack

Frontend: React + TypeScript + Vite, React Router, Tailwind CSS, Axios, TanStack Query, Leaflet or Mapbox GL JS for maps. Four role-based views (`/donor` , `/ngo` , `/driver` ,

/admin) share one frontend application rather than four separate apps.

Backend: Python + FastAPI. Chosen because the project is fundamentally an optimisation/data-science problem, which makes the algorithmic layer (candidate filtering, scoring, ranking, VRP) far easier to implement and test in Python than to bolt onto a different backend language.

Database: PostgreSQL + PostGIS. Locations are stored as `POINT(latitude longitude)` geometries rather than strings, enabling spatial queries such as "find NGOs within 10 km" via `ST_DWithin`.

Real-time layer: WebSockets for donation status, match results, NGO acceptance, driver assignment, live driver location, delivery status, and expiry warnings — the system should never rely on the client polling for updates.

Background jobs: Redis + Celery/RQ for expiry monitoring, automatic rematching, and queued matching computation under concurrent donation load.

Auth: JWT access tokens (short-lived) with refresh tokens; Argon2/bcrypt password hashing; role-based authorization (`DONOR` , `NGO` , `DRIVER` , `ADMIN`).

Deployment: Docker Compose bringing up frontend, backend, PostgreSQL, Redis, and the matching worker as one stack.

7. Data Model

7.1 Core tables

`users` (id, name, email, phone, password_hash, role, created_at) — `role` ∈ {`DONOR`, `NGO`, `DRIVER`, `ADMIN`}

`donors` (id, user_id, organisation_name, address, location, contact_person, verification_status, daily_waste_category, created_at)

`ngos` (id, user_id, organisation_name, address, location, storage_capacity_kg, available_capacity_kg, operating_start, operating_end, verification_status, created_at)

`ngo_food_categories` (ngo_id, food_category, accepted)

`ngo_demand` (id, ngo_id, food_category, required_quantity_kg, priority, valid_until, updated_at)

`donations` (id, donor_id, food_category, food_name, quantity_kg, prepared_at, available_from, expiry_time, pickup_location, special_requirements, status, created_at,

updated_at)

vehicles (id, driver_id, capacity_kg, current_location, availability_status)

deliveries (id, donation_id, ngo_id, driver_id, pickup_time, estimated_delivery_time, actual_pickup_time, actual_delivery_time, route_distance_km, estimated_duration_min, status)

handover_records (id, donation_id, delivery_id, donor_confirmation, ngo_confirmation, pickup_timestamp, delivery_timestamp, quantity_handed_over, donor_signature, recipient_signature, disclaimer_version, notes, created_at)

audit_logs (id, entity_type, entity_id, event_type, payload, record_hash, previous_hash, created_at)

7.2 Supporting tables added during technical review

refresh_tokens — keeps access tokens short-lived (15 min) without killing a driver's session mid-delivery.

notifications — persisted status-change records (matched, accepted, driver assigned, arriving, delivered) so a donor who was offline at match time still receives the update.

food_safety_info (donation_id, storage_temp_required, allergen_tags, packaging_type) — strengthens the shelf-life score and the audit trail beyond a bare timestamp.

ngo_verification_documents (ngo_id, document_type, file_url, uploaded_at) — backs the admin verification workflow with actual evidence (registration certificate, FSSAI Alliance membership number) instead of a bare toggle.

ratings_feedback — NGO ratings of donor food quality and donor ratings of pickup reliability, feeding a future "recurring donor" / "reliable NGO" score.

donation_photos (donation_id, photo_url, stage) — photo at pickup and at drop-off, strengthening the handover audit trail.

matching_weights_history (weights_version_id, city, food_category, w1..w5, created_at) — weights are versioned rather than mutated in place, so any historical match can still be explained by which weight set produced it.

7.3 Donation lifecycle (state machine)

```
AVAILABLE → MATCHING → MATCHED → ACCEPTED → DRIVER_ASSIGNED
           → PICKUP_STARTED → PICKED_UP → IN_TRANSIT
           → DELIVERED | PARTIALLY_DELIVERED
```

Failure branches:
MATCHING → NO_MATCH_FOUND
MATCHED/ACCEPTED → REJECTED → (loops back to MATCHING)
AVAILABLE|MATCHING|MATCHED → EXPIRED
DRIVER_ASSIGNED|PICKUP_STARTED → CANCELLED | DRIVER_ISSUE (reassign, same NGO)

PARTIALLY_DELIVERED matters because a driver may arrive with 35 kg while the NGO can only actually intake 28 kg that day; `quantity_handed_over` on the handover record is deliberately distinct from the donation's original `quantity_kg` so this is still recorded as a successful (partial) delivery rather than a failure.

7.4 Data integrity rules

`quantity_kg > 0 · expiry_time > available_from · storage_capacity_kg >= 0 · available_capacity_kg <= storage_capacity_kg`. NGO acceptance uses a row-level lock (`SELECT ... FOR UPDATE`) so two NGOs cannot simultaneously claim the same donation — the second caller receives `409 Conflict`, not a silent overwrite.

8. The Matching Algorithm

8.1 Formulation

For a donation *D* and a candidate recipient *N*, the system computes:

$$\text{Score}(D, N) = w_1 \cdot C + w_2 \cdot S + w_3 \cdot T + w_4 \cdot D + w_5 \cdot R$$

Factor	Meaning
C	Recipient capacity compatibility — can the NGO actually store and handle this volume?
S	Shelf-life compatibility — does remaining shelf life survive the whole handover?
T	Estimated transit-time compatibility — is the trip achievable within the usable window?
D	Recipient demand — how much does this recipient currently need?
R	Route/distance efficiency — how efficient is this leg across the wider dispatch plan?

Weights $w_1 \dots w_5$ are **tunable per city and per food category**, not fixed constants — this is explicit in the source brief and is why weights live in a versioned configuration table (`matching_weights_history`) rather than hard-coded values.

8.2 Step 1 — Hard-constraint filtering (before any scoring happens)

```
Food available?
    ↓
NGO eligible?
    ↓
Food category accepted?
    ↓
Enough capacity?
    ↓
NGO operational?
    ↓
Can pickup happen before expiry?
```

Only candidates that survive every one of these checks are scored at all. A donation of 40 kg with an 8:00 PM expiry is never scored against an NGO with 20 kg of capacity — it is rejected outright, before the weighted formula ever runs.

8.3 Step 2 — Feature calculation

- **Capacity (C):** $C = 1 - |capacity - quantity| / capacity$, clipped to [0,1] — rewards an appropriately-sized match, not just "big enough."
- **Shelf-life (S):** $S = (remaining_life - transit_time) / remaining_life$. A 120-minute remaining life with a 30-minute transit gives $S = 0.75$.
- **Transit (T):** $T = 1 - (ETA / ETA_max)$ — normalised so a 10-minute trip scores far higher than a 45-minute one.
- **Demand (D):** $D = \min(1, demand / donation_quantity)$ — an NGO needing 35 kg of a 40 kg donation outranks one needing only 5 kg.
- **Route (R):** derived from distance, ETA, live traffic, and the existing route plan — never simply $1 / distance$, since distance alone ignores traffic and vehicle availability.

8.4 Step 3 — Worked example

A 30 kg cooked-food donation in Koramangala, expiring in 2 hours, with three candidate NGOs:

--	--	--	--	--

NGO	Distance	ETA	Capacity	Demand
A	2 km	15 min	50 kg	10 kg
B	5 km	18 min	40 kg	30 kg
C	3 km	40 min	35 kg	30 kg

A naive "nearest NGO" system sends it to A. This system's candidate filtering and scoring instead recognise that A has low demand, B has high demand and a feasible ETA, and C has high demand but an inefficient ETA relative to the shelf-life window — so **B** is selected, even though it is not the closest option. This is the concrete demonstration of the project's central claim: not the nearest recipient, but the feasible recipient that minimises spoilage risk while maximising utilisation.

8.5 Step 4 — Rematching

If an NGO rejects or cancels after accepting, the system automatically excludes that NGO, recalculates candidates, reranks, and offers the next feasible recipient — the donor never has to manually restart the process.

8.6 Step 5 — Vehicle Routing Problem (advanced/stretch module)

With multiple concurrent donations and recipients live at once, the problem naturally extends toward a VRP formulation: one driver serving several pickups and drop-offs in sequence rather than one donation, one driver, one trip. This is treated as an advanced module built only after single-donation matching works reliably.

9. Routing & Real-Time Dispatch

Routing inputs explicitly include live traffic conditions, distance, estimated travel time, pickup time, the expiry window, and vehicle availability. For context, average travel time in Bengaluru runs about **3 minutes 37 seconds per kilometre**, against **2 minutes 4 seconds in free-flow conditions** (TomTom Traffic Index, 2025 data) — a gap large enough that ignoring live traffic in the routing score would materially misestimate feasibility.

Dispatch flow after NGO acceptance: find an available driver → check vehicle capacity → assign driver → calculate route → notify driver. The whole pipeline runs on real-time events (WebSocket-pushed), not polling: donation created → matching triggered → candidates calculated → best match selected → NGO notified → NGO accepts → driver assigned → route created → delivery tracked.

Background expiry monitoring runs continuously: a worker checks remaining time on every active donation, escalates priority as expiry approaches (>4h LOW , 2-4h MEDIUM , 1-2h HIGH , <1h CRITICAL), and marks a donation EXPIRED and un-dispatchable once remaining time hits zero.

10. Digital Handover, Traceability & Regulatory Context

10.1 What the sign-off actually is

When food is delivered, the system captures donor and NGO confirmation, quantity handed over, condition, and both parties' digital confirmation, producing an immutable audit record — optionally chained via `record_hash = SHA256(previous_hash + donation_id + timestamp + event_type)` , so any attempt to alter an earlier event breaks the chain.

10.2 What it is legally, and what it is not

India has no Good Samaritan food-donation statute. The digital sign-off is a documentation and traceability record under the **FSSAI Surplus Food Regulations, 2019** — it does not, and should not be presented as, granting legal immunity to donors or NGOs. This distinction should be stated explicitly wherever the sign-off is shown in the product (a `disclaimer_version` field keeps the exact text shown at each sign-off auditable).

10.3 A relevant regulatory driver

Under India's **Solid Waste Management Rules, 2026** (in force since 1 April 2026), bulk waste generators — including commercial kitchens producing more than 100 kg of waste per day — are made directly accountable for their own waste, and account for nearly **30% of total solid waste generation** (PIB, Ministry of Environment, Forest and Climate Change, 28 January 2026). This is a concrete compliance incentive for exactly the kind of donor this platform targets, and is why the `donors` table flags a `daily_waste_category` (`BULK_GENERATOR` VS `STANDARD`).

11. Impact, KPIs & Analytics

11.1 KPI categories (as specified in the brief)

- **Food:** kilograms diverted, meals recovered, share of donations successfully delivered.
- **Logistics:** average matching time, average delivery time, route distance saved, failed

pickups avoided.

- **Social:** organisations served, beneficiaries reached, recurring donors retained.

11.2 Formulas

<code>DeliveryRate</code>	<code>= successfulDeliveries / totalAssignedDeliveries × 100</code>
<code>AvgMatchTime</code>	<code>= Σ(matchTime - createdTime) / numberOfMatches</code>
<code>FoodDiverted</code>	<code>= Σ deliveredQuantity</code>
<code>FailureRate</code>	<code>= failedPickups / totalPickups</code>

11.3 Two derived metrics worth adding for a stronger demo

- **Route distance saved** = naive nearest-NGO distance – actual optimised-match distance, summed over a period. This is the number that concretely proves the matching engine beats a "just pick the closest NGO" baseline — the whole differentiator claimed in Section 3.2.
- **Spoilage-risk avoided** = count of donations where the shelf-life score was low (< 0.3) at match time but were still delivered successfully before expiry — framed as food a naive system would likely have failed to move in time.

11.4 An honest caveat

No figures are shown as achieved results anywhere in this project, because nothing has been built and run at scale yet. Any numbers presented are illustrative targets for what the KPIs *would* measure, not claims about outcomes already produced.

12. Minimum Viable Product

Rather than building every feature in Sections 5–11 simultaneously, the actual build target is:

1. Donor registration → create surplus → enter food info → submit.
2. NGO registration → set capacity → set accepted food types → set demand.
3. Algorithmic matching: donation → hard constraints → candidate NGOs → score → rank → best NGO.
4. Live routing: donor → NGO, with distance, ETA, route.
5. Acceptance: NGO accepts → driver assigned.

6. Tracking: pickup → transit → delivery.

7. Analytics: total kg rescued, successful deliveries, average matching time, average delivery time, NGOs served.

VRP and any ML module are explicitly advanced modules layered on top of this pipeline once it works end-to-end — not part of the MVP.

13. Scaling Roadmap

Single venue	→	Multiple venues	→	City-wide network	→	Multi-city
One kitchen, one recipient		Several donors in one zone		Bengaluru pilot across zones		Re-tuned matching weights per new city

Bengaluru is the proposed pilot city, chosen partly because usable local reference data already exists (the FSSAI Alliance's registered agency network, the TomTom traffic figures above) to seed and validate the matching and routing logic before expanding elsewhere.

14. Team Structure (6-Person Division)

Person	Ownership	Core responsibility
1	Backend + Database	Core API, PostgreSQL/PostGIS, authentication — the foundation everyone else's contract depends on
2	Donor Module	Donor UI + donation creation/lifecycle
3	NGO Module	NGO UI + capacity/demand/acceptance
4	Matching Engine	Constraint filtering + scoring algorithm — the project's technical differentiator
5	Routing + Driver	Maps, routing, driver tracking, dispatch
6	Admin + Analytics + Integration	Admin dashboard, KPIs, and final integration/deployment ownership

The division is by module, not by page or feature, specifically so all six can work in parallel without repeatedly touching the same files. A full API contract — exact request/response

JSON for every endpoint each person owns or consumes, plus mock fixtures so no one is blocked on Day 1 — governs the handoffs between modules and should be frozen by the end of Day 2, changeable only via reviewed pull request thereafter.

Integration is explicitly owned by Person 6 (lead) with Person 1 as co-lead — not by Person 4, whose time is protected for the algorithm that differentiates the project. Person 6's integration responsibilities include branch/PR hygiene, the mock-to-real cutover schedule, the end-to-end integration test spanning all six modules, Docker Compose assembly, deployment, and demo rehearsal on the deployed build.

15. What Makes This Project Technically Strong

The project is not impressive because it has a React frontend, maps, a login flow, and a dashboard — those are implementation details available to any team. The actual technical contribution is:

- **Constraint satisfaction** — who *can* receive this donation?
- **Multi-factor ranking** — who *should* receive it?
- **Time-sensitive optimisation** — who can receive it *before it becomes unusable*?
- **Dynamic routing** — what route gets it there efficiently, given live traffic?
- **Multi-agent coordination** — how are many donors, NGOs, and vehicles coordinated simultaneously?
- **Traceability** — can the system prove what happened to every donation, end to end?

The system's architecture and scoring algorithm are built around exactly these six concerns, and the presentation itself frames it that way: not simply finding the nearest NGO, but selecting a feasible recipient that minimises the risk of food becoming unusable while maximising utilisation.

16. Future Scope (explicitly beyond the core system)

- Demand prediction for non-profits, using historical donation/demand data by NGO, category, day, and time.
- ML-based spoilage and expiry prediction, replacing a manually entered expiry with a predicted safe-use window based on food category, temperature, preparation time, packaging, and elapsed time.

- Dynamic multi-vehicle routing (full VRP at production scale).
 - A volunteer driver marketplace, replacing the simulated driver layer used in the prototype.
 - Predictive surplus forecasting.
 - Integration with municipal food-waste systems, relevant given the SWM Rules, 2026 bulk-generator accountability requirement described in Section 10.3.
-

Sources cited in this document

- UNEP Food Waste Index Report 2024 (2022 data), for global and Indian household food-waste figures.
- Government of India, Lok Sabha Unstarred Question 2181, 12 March 2025, for the Indian household waste confirmation and the FSSAI Indian Food Sharing Alliance registered-agency count.
- FSSAI Surplus Food Regulations, 2019, for the legal basis of the digital sign-off/documentation record.
- India's Solid Waste Management Rules, 2026 (in force 1 April 2026) and PIB, Ministry of Environment, Forest and Climate Change (28 January 2026), for the bulk-waste-generator accountability figures.
- TomTom Traffic Index, 2025 data, for Bengaluru average vs. free-flow travel time.